UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
CURSO: MANEJO E IMPLEMENTACIÓN DE ARCHIVOS

MANUAL TECNICO
Sistema de Gestión Documental

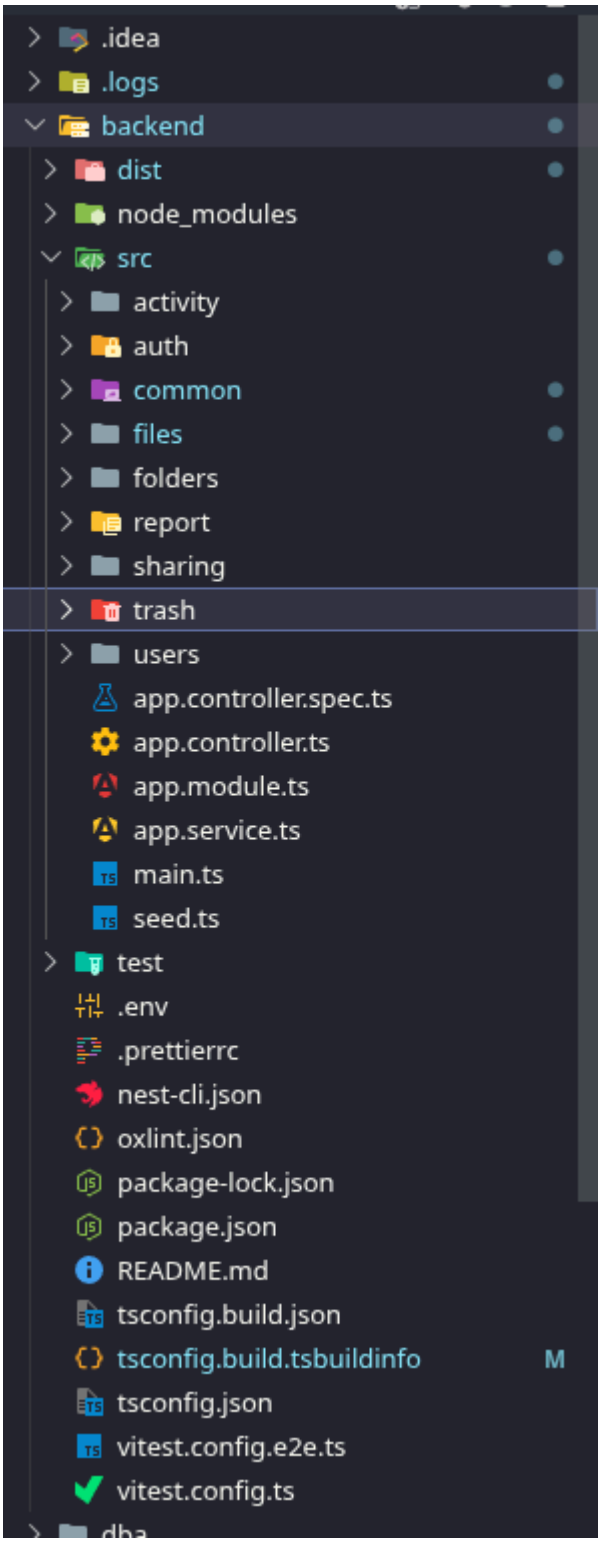
ESTUDIANTE **CARNÉ:**
Cristian Alejandro Roldán López **202147280**

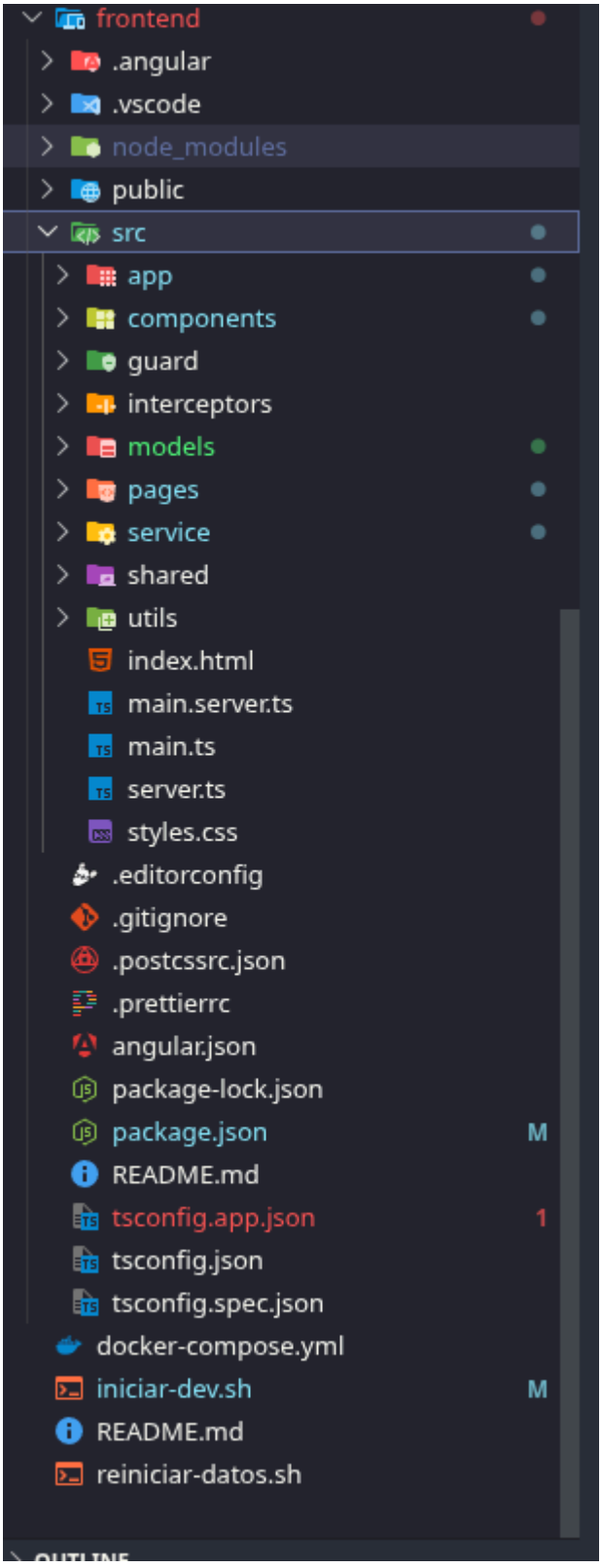
QUETZALTENANGO, 23 DE OCTUBRE 2,026

Tecnologías utilizadas

- **Versión de Node:** 22.20.0
- **NestJs 12**
- **Angular 21**
- **Mongoose**
- **bycriptjs**

Distribución del proyecto.




frontend
.angular
.vscode
node_modules
public
src
app
components
guard
interceptors
models
pages
service
shared
utils
index.html
main.server.ts
main.ts
server.ts
styles.css
.editorconfig
.gitignore
.postcssrc.json
.prettierrc
angular.json
package-lock.json
package.json
M
README.md
tsconfig.app.json
1
tsconfig.json
tsconfig.spec.json
docker-compose.yml
iniciar-dev.sh
M
README.md
reiniciar-datos.sh

La arquitectura utilizada en el backend fue por capas.

- controller: expone las rutas REST y delega en el servicio.
- service: contiene la lógica de negocio y las reglas de validación.
- schemas: esquemas de Mongoose que definen las colecciones y sus índices.
- dto: objetos de entrada validados con class-validator.
- module: registra los modelos, controladores y servicios del módulo.

AuthService busca el usuario, compara la contraseña con bcryptjs y verifica que la cuenta esté activa.

Si todo es correcto, firma un token JWT con el identificador del usuario (sub) y su rol (rol), con una duración de JWT_EXPIRES_IN segundos.

El frontend guarda el token y lo envía en cada petición con el encabezado Authorization: Bearer.

JwtAuthGuard: se aplica a todas las rutas y exige un token válido, salvo las marcadas como públicas.

RolesGuard: compara el rol del token con los roles indicados en el decorador @Roles de cada ruta.

La validación de entrada usa ValidationPipe con whitelist, que elimina los campos no declarados en el DTO. El identificador de cada ruta se valida con ParseObjectIdPipe.

Carga inicial y datos de demostración

Al arrancar, si no existe ningún usuario, el backend crea el administrador con las variables ADMIN_* del archivo .env.

El comando npm run seed (o npm run seed:reset para regenerar) carga datos de demostración: tres usuarios de prueba, carpetas y subcarpetas, 50 archivos, más de 20 comparticiones con los tres permisos, elementos en la papelera, restauraciones, una eliminación definitiva y registros de actividad distribuidos en los últimos 40 días.

seed:reset borra y regenera los datos; es destructivo y no debe usarse con datos reales.

Variables de entorno

PUERTO	**Puerto del backend (por defecto 3000).**
MONGO_URI	**Cadena de conexión a MongoDB. El nombre final es la base de datos.**
ORIGEN_CLIENTE	**Origen permitido por CORS http://localhost:4200).**
JWT_SECRET	**Clave para firmar los tokens. Debe ser larga y aleatoria.**
JWT_EXPIRES_IN	**Duración de la sesión en segundos (86400 = 24 horas).**
ADMIN_FIRST_NAME, ADMIN_LAST_NAME, ADMIN_EMAIL, ADMIN_PASSWORD	**Datos del administrador inicial.**

La base de datos esta en un contenedor docker

```
services:
 mongo:
   image: mongo:7
   container_name: gestor-mongo
   restart: unless-stopped
   ports:
     - "127.0.0.1:27017:27017"
   volumes:
     - mongo_datos:/data/db

volumes:
 mongo_datos:
```

Para iniciar la app esta un script que se llama iniciar-dev.sh
./iniciar-dev.sh